01 OPEN YOUR FIRST NOTEBOOK

Getting started with JupyterLab

A practical introduction with NumPy, SciPy and Matplotlib

Allow about 35–45 minutes. You only need a computer with a keyboard, an up-to-date web browser, an internet connection and basic Python knowledge. You will create a notebook, run calculations, draw a figure and keep a copy of your work.

> **Where to practise**
> Open the official demo: **jupyter.org/try-jupyter/lab/**. No account or local installation is needed. This demo uses JupyterLite, which runs Python inside your browser.
> **This is a practice environment.** Your actual exams take place on the course's own server. Follow the instructor's directions for exam access and submission.

Create a notebook

1. Let the page finish loading. You may see an introductory notebook; you can leave it open or close its tab.
2. Click the **+** above the file list on the left to open the **Launcher**. Under **Notebook**, choose a **Python** kernel. Its label may include **XPython** or **Pyodide**; do not choose a different programming language.
3. Right-click the new file in the file list, choose **Rename**, and call it `jupyter_practice.ipynb`. Keep the `.ipynb` extension: it identifies a Jupyter notebook.
4. Click in the empty cell, type the code below, then press `Shift+Enter`. Wait for the first imports to finish.

Code cell 1 — import the libraries

```
import numpy as np
import scipy
from scipy.integrate import trapezoid
import matplotlib
import matplotlib.pyplot as plt

print("NumPy:", np.__version__)
print("SciPy:", scipy.__version__)
print("Matplotlib:", matplotlib.__version__)
```

Three version numbers should appear below the cell. Their exact values may change. The demo's Python environment includes all three libraries; you do not need to install them for this guide.

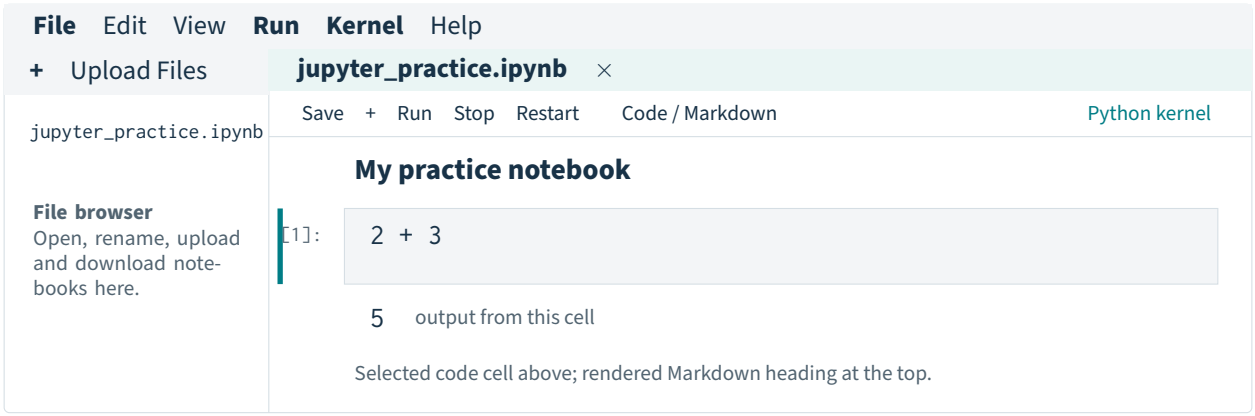
First checkpoint: you have a named notebook and a cell that runs. If you see an import error, check that the notebook is using a Python kernel and try restarting it (page 3).

Route through the guide: interface and cells (2) • kernel and execution order (3) • calculations (4) • plots (5) • upload and download (6) • files and recovery (7) • independent practice (8).

02 THE INTERFACE AND ITS CELLS

Find your way around

The notebook is a sequence of **cells**. Select a cell to work on it; its output appears immediately below it.



Simplified interface map. Icons, labels and spacing may vary.

Use the two cell types

Code cells execute Python. Add a cell below your imports using the notebook toolbar's **+** button. Type `2 + 3` and run it with `Shift+Enter`; the output is `5`.

Markdown cells contain explanations, headings and equations. Add another cell, change **Code** to **Markdown** using the toolbar drop-down, then type this:

```
# My practice notebook
I am using **NumPy**, **SciPy** and **Matplotlib**.
An equation: $y = \sin(x)$.
```

Run it with `Shift+Enter` to format the text. Double-click the formatted text to edit it again. Markdown is explanatory text; it does not define Python variables.

Switch between typing and arranging cells

Edit mode: click inside a cell or press `Enter` to type. **Command mode:** press `Esc` to act on the whole cell. In command mode, `B` inserts a cell below, `A` inserts one above, `M` changes it to Markdown and `Y` changes it to Code. These letters are not shortcuts while you are typing inside a cell.

> **Try it**
>
> Create a temporary cell, enter `10 * 2` and run it. Delete that cell using its context menu's **Delete Cells** action, then use **Edit > Undo Cell Operation** to restore it. You can also press `Esc`, then `Z`, to undo a cell deletion.

03 THE KERNEL AND EXECUTION ORDER

Understand what is running

The **notebook** stores your cells and any saved outputs. The **kernel** runs the code and holds variables in memory. All code cells in one notebook normally share that memory.

Run this small experiment

Add the following three code cells *below* the cells already in your practice notebook. Run them in order.

Cell A

```
a = 10
```

Cell B

```
b = a + 1
```

Cell C

```
print(a, b)
```

The output is `10 11`. Now change Cell A to `a = 20` and run **only A and C**. You get `20 11`: Cell B has not run again, so `b` still contains its earlier value. Run B, then C, to obtain `20 21`.

> **Editing a cell does not execute it**
> After changing a cell, run it again *and* rerun any later cells that depend on it. Existing outputs do not update automatically. The numbers beside code cells show execution order, which can differ from page order.

Read the execution indicator

`[ ]`	The cell has no execution count yet.
`[*]`	The cell is executing or waiting for the kernel.
`[7]`	This was the kernel's seventh numbered execution.

An assignment such as `a = 10` normally produces no output. Use `print(a)` or put `a` on the last line to display it.

Restart and check the whole notebook

Open **Kernel > Restart Kernel and Run All Cells...** and confirm. The kernel forgets its variables, then executes the notebook from the top. If the command is labelled differently, use **Restart Kernel** followed by **Run > Run All Cells**. This checks that all the code you need is actually present and in the right order.

Restarting alone does not delete your cell text. Saved outputs can remain visible even though their variables no longer exist in memory. If a calculation takes too long, try **Kernel > Interrupt Kernel**. If the browser kernel does not respond, try restarting it; avoid running the same slow cell repeatedly.

04 ONE CELL AT A TIME

Calculate with NumPy and SciPy

Continue in the same notebook. Keep the imports from page 1 at the top. Each block below goes into a **separate Code cell**. Copy the code only; do not copy the expected output.

NumPy: work with a whole array

```
values = np.array([1.0, 2.0, 3.0, 4.0])
print(values**2)
print(values.mean())
```

Expected output:

```
[ 1.  4.  9. 16.]
2.5
```

`values**2` squares every element. Python uses `**` for powers. Array indexing starts at zero: `values[0]` is `1.0`.

Build sample points for a sine curve

```
x = np.linspace(0, np.pi, 201)
y = np.sin(x)
print(x.shape, y.shape)
```

Expected output: `(201,) (201,)`. The arrays contain 201 matching sample points, including both endpoints. NumPy's trigonometric functions use radians. The variables `x` and `y` remain available to the following cells.

SciPy: approximate the area under the curve

```
area = trapezoid(y, x=x)
print(f"Integral estimate: {area:.6f}")
print(f"Absolute error: {abs(area - 2):.6f}")
```

Expected output:

```
Integral estimate: 1.999959
Absolute error: 0.000041
```

Here `trapezoid` is the function imported from `scipy.integrate`. It approximates an integral from sampled data. The exact comparison is

$$\int_0^{\pi} \sin(x)\,\mathrm{d}x = 2.$$

The formatting `:.6f` displays six digits after the decimal point; it does not round the value stored in `area`.

> **Make a change and follow its effects**
>
> Replace `201` by `21` in the sampling cell. Rerun that cell and the integration cell. Compare the error, then restore `201` and rerun both. Merely changing the number in the editor is not enough.

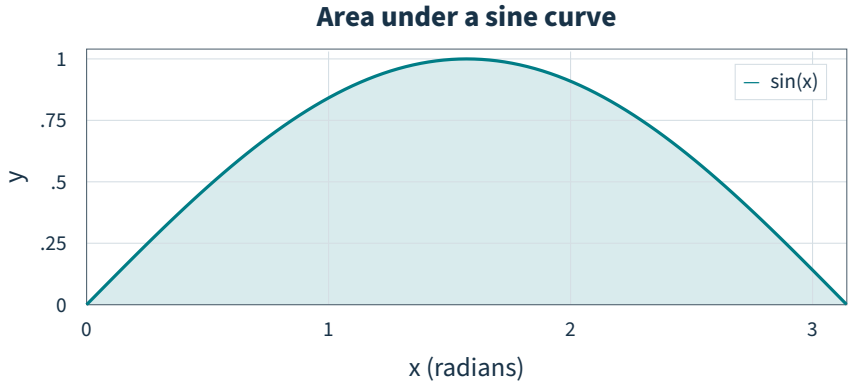
05 MATPLOTLIB AND MARKDOWN

Draw and explain a figure

Add a Code cell after the integration cell. It uses x and y from page 4, so run their definition first.

```
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(x, y, label="sin(x)")
ax.fill_between(x, y, alpha=0.15)
ax.set_xlabel("x (radians)")
ax.set_ylabel("y")
ax.set_title("Area under a sine curve")
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
plt.show()
```

Run the cell. A figure like this should appear **below the code**, inside the notebook.



Explain your result in a Markdown cell

Add a cell below the plot, change its type to **Markdown**, enter the following, then press `Shift+Enter`.

```
## Numerical integration
The estimated area is approximately **2**.
The exact result is $\int_0^{\pi} \sin(x)\,dx = 2$.
```

Practise updating a figure

Change the plot title and rerun the plotting cell. Then change the sample count on page 4, rerun that sampling cell, the integration cell and the plotting cell. Each run creates a fresh figure because the cell begins with `plt.subplots()`.

Checkpoint: you can combine executable code, numerical output, a labelled figure and a written explanation in one notebook.

06 MOVE NOTEBOOKS, DATA AND FIGURES

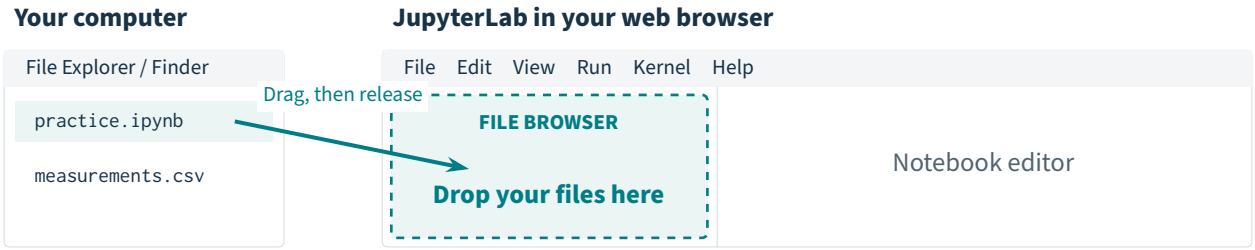
Upload and download files

Upload copies a file from your computer into JupyterLab. **Download** copies a file from JupyterLab to your computer. These actions work for notebooks (`.ipynb`), data and figures.

Upload an existing notebook

1. Open the folder containing the `.ipynb` file in your computer's file manager (File Explorer or Finder). Place this window beside your browser.
2. In JupyterLab's left sidebar, open the **file browser** and select the destination folder.
3. Click and hold the local file, drag it onto the **file list in JupyterLab's left sidebar**, then release. You can select several files to upload together.
4. Wait for its name to appear, then double-click it to open. Select a Python kernel if asked.

Or click **Upload Files** (the upward arrow above the file list) and choose the notebook.



Simplified diagram: the shaded file list on the left is the upload target.

Download your notebook

1. Save your latest notebook edits: use the Save button or `Ctrl+S` (`Cmd+S` on macOS).
2. In the left file browser, **right-click your notebook's `.ipynb` filename and choose Download**.
3. Find the `.ipynb` copy in Downloads, or the folder chosen in your browser. Upload it again to reopen and continue later.

Download data and figures the same way (examples on page 7).

Upload and read a data file

Upload the supplied `measurements.csv` to the **same folder as your notebook**, using the button or drag and drop. In a new Code cell, run:

```
import numpy as np
data = np.loadtxt("measurements.csv", delimiter=",", skiprows=1)
print(data.shape)
```

Expected output: `(5, 2)` — five data rows and two columns. `np.loadtxt` reads the data; `skiprows=1` skips the header.

> **Save and download are different actions**
>
> Save writes to browser storage; Download keeps a copy on your computer. Download your work before leaving: clearing site data, using private browsing or switching browsers can make the demo's stored files unavailable.

07 FILES AND TROUBLESHOOTING

Keep your results and recover from errors

Create files to download

After running the sine-curve cells on pages 4–5, run this Code cell. It saves the figure and a CSV containing the plotted data.

```
fig.savefig("sine_curve.png", dpi=150, bbox_inches="tight")
np.savetxt("sine_data.csv", np.column_stack((x, y)),
           delimiter=",", header="x,y", comments="")
```

The files are written to the kernel's current folder, normally the notebook's folder in this demo. Refresh the file browser if needed, then right-click each filename and choose **Download**. Saving a file from Python does not automatically download it to your computer.

Reopen a saved notebook

Save and download your practice notebook. Rename the copy to `reopened.ipynb` on your computer, then upload and open it as described on page 6. Upload `measurements.csv` to the same folder too, before restarting the kernel and running all cells.

> **Keep the input files with your notebook**
> A notebook contains code and saved outputs, not live Python variables or copies of the data files its code reads. A relative filename such as `measurements.csv` is resolved from the kernel's current folder; Python in the demo cannot read your computer's Downloads folder directly.

When something goes wrong

What you see	What to try
`FileNotFoundError`	Check the exact filename and its extension, and upload the file to the notebook's folder. Use `import os`, then `print(os.getcwd())` and `print(os.listdir())` to inspect the kernel's current folder and files.
`NameError`	Read the variable name in the last line of the error. Check its spelling and run the cell that defines it. For `np`, `plt` or `trapezoid`, rerun the imports.
`SyntaxError` or `IndentationError`	Check the highlighted line, brackets, quotes, colons and indentation. Fix the code and rerun the cell.
`TypeError` or `ValueError`	Read the last line of the traceback. Check the function's inputs and the shapes of your arrays.
A cell stays at `[*]`	Allow time for initial loading. For a stuck calculation, try **Kernel > Interrupt Kernel**, then restart if necessary.
Old or unexpected output	Rerun the cells that define the data and then the cells that use it. If uncertain, restart and run all.
An import fails	Confirm that a Python kernel is selected, then restart and rerun the imports. If it still fails, ask the instructor.

Getting help: type `np.sin?` in a Code cell and run it to request function help. You can also use `help(np.sin)`. Press `Tab` after part of a name, such as `np.lin`, to request completion; `Shift+Tab` inside a function call requests its tooltip.

08 FINAL PRACTICE AND QUICK REFERENCE

Try it on your own

Allow about 10 minutes for this final check. Create a new Python notebook named `final_practice.ipynb`. Refer to earlier pages when you need them.

1. Add a Markdown heading and one sentence explaining the task.
2. Import NumPy, Matplotlib and `trapezoid` from SciPy.
3. Create 101 equally spaced points from 0 to 1 and calculate $y = x^2$. Remember: Python writes the square as `x**2`.
4. Approximate the integral with `trapezoid(y, x=x)`. Display the estimate and its absolute error relative to $1/3$.
5. Plot the curve with a title and labels for both axes.
6. Restart the kernel and run all cells. Resolve any errors, save the notebook and download it to your computer.

Check your result: the integral estimate is approximately `0.333350`; the absolute error is approximately `0.000016667`. The curve rises from $(0, 0)$ to $(1, 1)$.

Useful keyboard shortcuts

Click inside the notebook before using these shortcuts. In the table, "Esc, then" means press and release Esc first.

Keys	Action
`Shift+Enter`	Run the selected cell and select the next cell.
`Ctrl+Enter`	Run the selected cell and stay on it.
Esc, then `A` / `B`	Insert a cell above / below.
Esc, then `M` / `Y`	Change the cell to Markdown / Code.
`Enter` / `Esc`	Enter edit mode / command mode.
Esc, then `Z`	Undo the most recent cell operation.
`Ctrl+S`	Save the active notebook.

On macOS, use `Cmd` in place of `Ctrl`. Menu commands provide an alternative if a browser or keyboard intercepts a shortcut.

For graded work, keep provided cells and metadata, and follow the required names, arguments and return values. Use the course server and the instructor's submission procedure for exams.

Official references JupyterLab interface • Notebooks • Files • Commands • JupyterLite
NumPy basics • SciPy trapezoid • Matplotlib quick start • Demo's Python packages
Reading CSV data • Files in browser kernels.

Prepared October 2026. Menu labels and package versions may change.